



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Análisis y Diseño de Sistemas.

Sistema Embebido Distribuido: Detección de Obstáculos con Ultrasonido + Visión

Alumnos:

Joshua Osorio O. – 1293271

Docente:

Fernando E. Saucedo L.

Marzo/2026

Contenido

Bibliografía.....¡Error! Marcador no definido.

Hardware (sistema embebido) Identificación de material

- Microcontrolador: Arduino UNO / ESP32
- Sensor ultrasónico: HC-SR04
- Driver de motores: L298N
- Motores DC + chasis
- Fuente de alimentación
- Módulo de comunicación (USB o Bluetooth HC-05)

Software (PC)

- Computadora
- Cámara (webcam o USB)
- Lenguaje: Python
- Librerías:
- OpenCV (visión por computadora)
- PySerial (comunicación con Arduino)

División del sistema

Función	Responsable	Justificación
Medición de distancia (ultrasonido)	Hardware (Arduino)	Requiere lectura en tiempo real y baja latencia directamente desde el sensor
Captura de imagen	Software (PC)	La cámara se procesa mejor en un sistema con mayor capacidad de cómputo

Procesamiento de visión (detección de obstáculos)	Software (PC)	Algoritmos de visión (OpenCV) requieren alto procesamiento y flexibilidad
Toma de decisiones (avanzar, detener, esquivar)	Software (PC)	Permite modificar fácilmente la lógica y aplicar algoritmos más complejos
Control de motores	Hardware (Arduino)	Necesita respuesta inmediata y control preciso mediante PWM
Comunicación (envío/recepción de datos)	Ambos (Arduino + PC)	Se requiere coordinación para enviar datos de sensores y recibir comandos

Asignación

Función	Componente asignado	Detalles
Medición de distancia (ultrasonido)	Arduino UNO	Sensor HC-SR04 conectado a pines digitales (Trigger/Echo), medición mediante pulsos
Captura de imagen	PC (Python + OpenCV)	Uso de cámara USB/webcam con <code>cv2.VideoCapture()</code>
Procesamiento de visión (detección de obstáculos)	PC (Python)	Uso de OpenCV para detección de objetos (contornos, color, etc.)
Toma de decisiones	PC (Python)	Algoritmo que combina distancia + visión para decidir acciones ('F','L','R','S')
Comunicación con Arduino	Arduino + PC (Serial USB/Bluetooth)	Envío de datos con <code>Serial.print()</code> y recepción con <code>Serial.read()</code>
Control de motores (PWM/dirección)	Arduino UNO	Driver L298N, control de velocidad con PWM y dirección con pines digitales
Accionamiento de motores	Driver L298N + Motores DC	El driver recibe señales del Arduino y controla los motores del robot

Arduino UNO:

- Código C++ usando `setup()` y `loop()`
- Lectura del sensor ultrasónico cada 50 ms
- Cálculo de distancia (Trigger/Echo)
- Envío de distancia vía `Serial.print()`

- Recepción de órdenes: 'F' (forward), 'L' (left), 'R' (right), 'S' (stop)
- Control de motores vía PWM (funciones como avanzar(), girarIzq(), etc.)

Software en PC (Python):

- Uso de pyserial para comunicarse con Arduino
- Uso de OpenCV para captura de cámara
- Algoritmo de decisión combinando distancia + visión
- Detección de obstáculos (contornos, color o forma)
- Posibilidad de mostrar video en tiempo real
- Registro de datos en archivos (.txt o .csv)

Flujo de comunicación

1. Arduino mide distancia (ej: 15 cm)
2. Envía dato por Serial: "15"
3. PC captura imagen y detecta obstáculo
4. PC decide acción:
 - Distancia < 10 → detener
 - Obstáculo detectado → esquivar
 - Sin obstáculo → avanzar
5. PC envía comando al Arduino:
 - 'F' → avanzar
 - 'L' → izquierda
 - 'R' → derecha
 - 'S' → stop
6. Arduino ejecuta movimiento

Código a implementar en Arduino

```
char command;

void setup(){
  Serial.begin(9600);
}

void loop(){
  long distancia = medirDistancia(); // función ultrasónico
  Serial.println(distancia);

  if (Serial.available()){
    command = Serial.read();
    switch (command){
      case 'F':
        avanzar();
        break;
      case 'L':
        girarIzq();
        break;
      case 'R':
        girarDer();
        break;
      case 'S':
        detener();
        break;
    }
  }
}
```

Código a implementar en Python

```
import serial
import cv2

arduino = serial.Serial('COM3', 9600)
cap = cv2.VideoCapture(0)

def decidir(distancia, obstaculo, izquierda):
    if distancia < 10:
        return 'S'
    elif obstaculo:
        return 'R' if izquierda else 'L'
    else:
        return 'F'

while True:
    distancia = int(arduino.readline().decode().strip())

    ret, frame = cap.read()
    obstaculo = detectar_objeto(frame) # función de visión
    izquierda = detectar_posicion(frame)

    comando = decidir(distancia, obstaculo, izquierda)
    arduino.write(comando.encode())
```

Métodos de verificación sugeridos

Simulación en software:

- Probar función decidir() con distintos valores de distancia
- Simular detección de objetos (True/False)

Pruebas de comunicación:

- Enviar valores manuales desde Arduino
- Verificar respuesta en Python

Simulación de entorno:

- Usar videos o imágenes en OpenCV
- Simular obstáculos

Pruebas parciales:

- Solo sensor ultrasónico
- Solo cámara
- Solo motores

Integración final:

- Evaluar comportamiento completo del robot

Criterios funcionales de verificación

- ¿Se mide correctamente la distancia?
- ¿Se detectan objetos con la cámara?
- ¿Se toman decisiones correctas?
- ¿Arduino ejecuta correctamente los comandos?
- ¿El robot evita obstáculos correctamente?